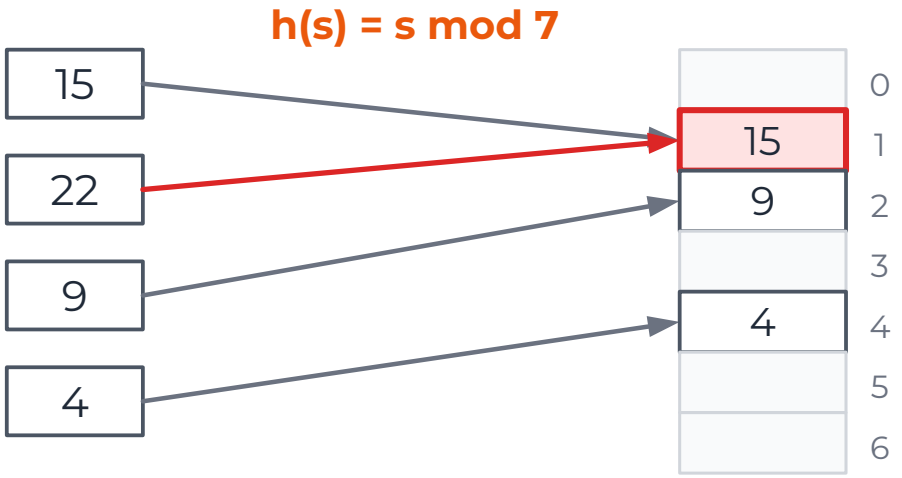
Hashverfahren

Dictionaries - Streuspeicherung

Grundidee



Ziel

Zugriff in **konstanter Zeit**

- **Hashtabelle** mit m Zellen, **Hashfunktion** h(s) ∈ {0, …, m−1}
- Schlüssel s liegt (idealerweise) direkt auf Platz **h(s)**
- **Kollision**: $h(s_1) = h(s_2)$ für $s_1 \neq s_2$ (hier: 15 und 22)
- m ⋘ Anzahl möglicher Schlüssel → Kollisionen **nicht vermeidbar**

Hashfunktion: Division

- **Division**: h(s) = s mod m

Schlüssel s	2	4	12	14	22
s mod 10	2	4	2	4	2
s mod 11	2	4	1	3	0

Wahl von m

Primzahl, nicht nahe an einer Zweierpotenz ($m \neq 2^k$, 10^k, 2^k-1)

- **m = 10**: 2, 12 und 22 landen alle auf Platz 2 (Kollisionen)
- **m = 11** (Primzahl): alle Schlüssel verteilen sich auf verschiedene Plätze
- S = {2, 4, ..., 200}: m = 100 → 50 Kollisionen, m = 101 → keine

Hashfunktion: Multiplikation

- **Multiplikation**: $h(s) = \lfloor m \cdot \{s \cdot c\} \rfloor$ mit $0 < c < 1$
- Beste Verteilung für **$c = (\sqrt{5} - 1) / 2 \approx 0{,}618$**

Beispiel: m = 64, c = 0,618

s	s · c	{s · c}	m · {s · c}	h(s)
4	2,472	0,472	30,208	**30**
5	3,090	0,090	5,760	**5**
6	3,708	0,708	45,312	**45**

Notation

{x} = gebrochener Anteil

⌊x⌋ = ganzzahliger Anteil

- m darf eine **Zweierpotenz** sein: $m = 2^p$
- **Ganzzahl-Arithmetik**: $h(s)$ = obere p Bits von $(s \cdot \tilde{c} \bmod 2^w)$, $\tilde{c} \approx c \cdot 2^w$

Kollisionen sind unvermeidbar

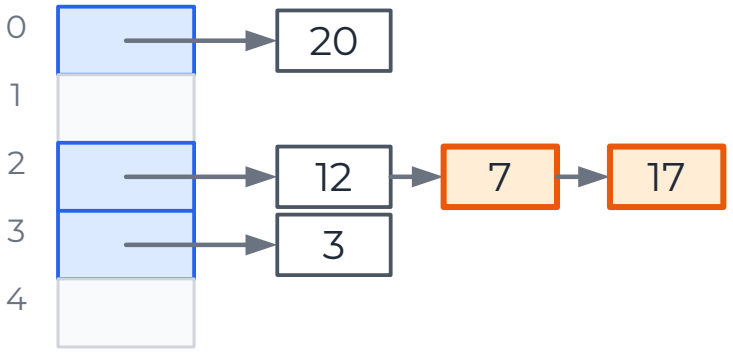
- m ⋘ Anzahl möglicher Schlüssel → Kollisionen **unvermeidbar**
- Schon bei wenigen Einträgen sehr wahrscheinlich (**Geburtstagsparadoxon**)

Tabellengröße m	Einträge k für ≥ 50 %
365	23
1 000	≈ 38
1 000 000	≈ 1 180

Fazit

Kollisionen **akzeptieren** und bei Auftreten **behandeln** (Kollisionsauflösung)

Kollisionsauflösung: Verkettung



Beispiel

m = 5, h(s) = s mod 5

Einfügen: 12, 7, 20, 3, 17

12, 7 und 17 → Platz 2: Liste wird länger

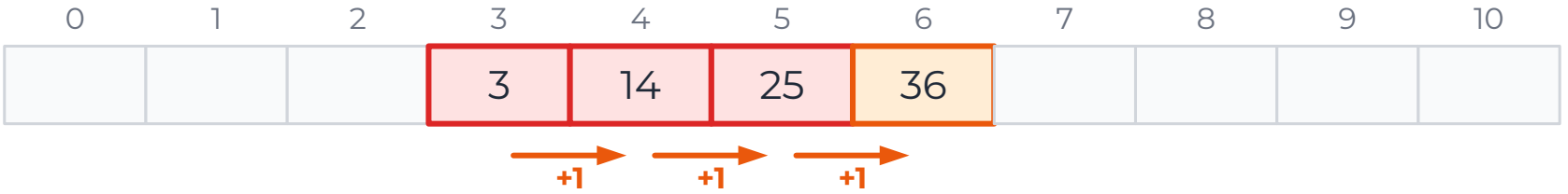
- Jeder Tabellenplatz verweist auf eine **verkettete Liste** („Array von Listen“)
- Kollision: Schlüssel **an die Liste anhängen** – Suchen und Löschen nur in dieser Liste
- Auch **Offenes Hashing** genannt (Variante: fester Überlaufbereich)

Kollisionsauflösung: Offene Adressierung

Sondierverfahren	Sondierfolge $i(s)_j$, $j = 0, \ldots, m-1$
Linear	$(h(s) + j) \bmod m$
Quadratisch	$(h(s) + (-1)^{j+1} \cdot \lceil j/2 \rceil^2) \bmod m$
Doppelhashing	$(h(s) + j \cdot h^*(s)) \bmod m$

- Kollision → Schlüssel kommt auf einen **anderen freien Platz** der Tabelle (Geschlossenes Hashing)
- **Sondierfolge** $i(s)_0$, $i(s)_1$, ...: Plätze, die nacheinander geprüft werden
- Sie beginnt immer bei $\mathbf{i(s)_0 = h(s)}$
- Anforderung: **schnell berechenbar** und durchläuft **alle Plätze**

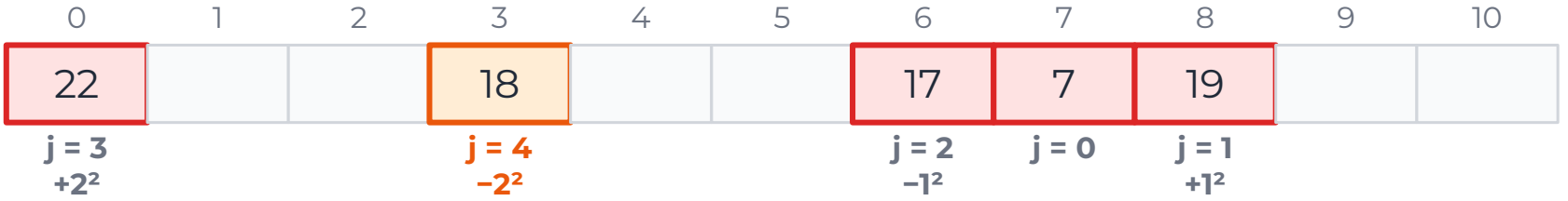
Lineares Sondieren



m = 11, h(s) = s mod 11, Einfügen von 36: h(36) = 3

- $i(s)_j$ = **(h(s) + j) mod m**: immer der **nächste Platz**
- Problem: **primäre Clusterbildung** – belegte Blöcke wachsen immer weiter

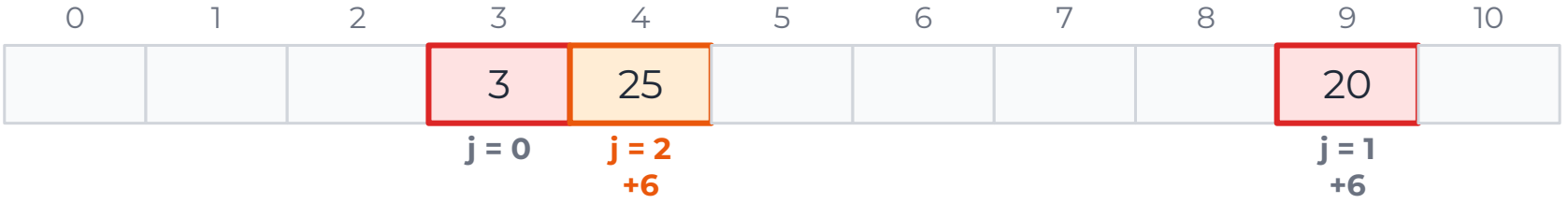
Quadratisches Sondieren



m = 11, h(s) = s mod 11, Einfügen von 18: h(18) = 7

- $i(s)_j$ = **$(h(s) + (-1)^{j+1} \cdot \lceil j/2 \rceil^2) \bmod m$**
- Abwechselnd **+1², −1², +2², −2², …** zum Hashwert addieren
- Beispiel s = 18: Folge **7, 8, 6, 0, 3** → 18 kommt auf Platz 3
- Alle Plätze erreichbar, wenn **m Primzahl** mit **m = 4k + 3** (7, 11, 19, 23, …)

Doppelhashing



m = 11, h(s) = s mod 11, h*(s) = 1 + (s mod 10), Einfügen von 25: h(25) = 3, h*(25) = 6

- $i(s)_j$ = **(h(s) + j · h*(s)) mod m** mit h*(s) ∈ {1, …, m−1}
- **Schrittweite hängt vom Schlüssel ab** (linear: immer 1) → kollidierende Schlüssel gehen meist verschiedene Wege
- Beispiel s = 25: Folge **3, 9, 4** → 25 kommt auf Platz 4
- Alle Plätze erreichbar, wenn **h*(s) und m teilerfremd** (z. B. m Primzahl)

Offene Adressierung: Suchen & Löschen

Markierung	Bedeutung
belegt	Platz enthält einen Schlüssel
frei	auf keiner Sondierfolge → Suche **stoppt**
gelöscht	kein Schlüssel → Suche geht **weiter**

Warum „gelöscht“?

Mit „frei“ würde die Suche zu früh enden.



m = 11, alle h(s) = 3, nach delete(14)

- **search(25)**: 3 ≠ 25 → 4 gelöscht, **weiter** → 5 gefunden ✓
- **insert** darf auf „gelöscht“ und „frei“ schreiben

Rehashing

1. Neue Tabelle mit größerem **m'** anlegen (üblich: $m' \approx 2 \cdot m$)
2. Alle Schlüssel mit der neuen Hashfunktion **neu einfügen**
3. Alte Tabelle löschen

Schlüssel s	h(s), m = 5	h'(s), m' = 11
12	2	1
7	2	7
20	0	9
3	3	3